



Guía de Estudio y Arquitectura

FitZone Sports: Roadmap didáctico y técnico del backend con NestJS

Esta guía funciona como un **roadmap didáctico y técnico** para dominar el backend de **FitZone Sports**, combinando conceptos de ingeniería de software con explicaciones y fragmentos reales de código fuente.

Mapa de Ruta del Aprendizaje

1 Fundamentos & Arquitectura en Capas y DI	2 Ciclo de Petición HTTP (Pipe → Ctrl → Svc → Repo)
3 El Viaje del Usuario (Registro → Socio → Membresía)	4 Persistencia y Transacciones (Prisma & Atomicidad)
5 Manejo de Errores Estandarizado (RFC 9457)	6 Visión a Futuro (Mediador & Hexagonal M5)

1. Fundamentos y Filosofía de Arquitectura

El sistema está desarrollado en **NestJS** (Node.js/TypeScript) conectado a PostgreSQL mediante **Prisma ORM**.

1. Separación de Responsabilidades (Capas)

Los módulos se dividen en **Controllers** (transporte HTTP y rutas), **Services** (reglas de negocio), **Repositories** (acceso a datos), **Entities** (dominio puro) y **DTOs** (validación).

2. Inversión de Dependencias (DIP) con Tokens

Los servicios dependen de interfaces (ej. `UsuarioRepository`) y no de Prisma directamente. Esto permite aislar la lógica de negocio y facilita el testeo unitario mediante mocks.

3. Patrón Data Mapper

Las tablas relacionales de la base de datos se traducen a entidades de dominio independientes a través de los repositorios, desacoplando el esquema SQL de las reglas del negocio.

2. Anatomía de una Petición HTTP (Request Lifecycle)

Toda petición atraviesa una tubería estricta antes de impactar en la base de datos:

1. **Cliente** envía petición JSON (ej. POST /api/v1/usuarios).
2. **ValidationPipe** valida tipos y restricciones. Si falla, intercepta y retorna 422 Unprocessable Entity.
3. **Controller** recibe el DTO limpio, ejecuta la acción y configura cabeceras HTTP de respuesta.
4. **Service & Repository** aplican reglas de negocio y ejecutan consultas atómicas en PostgreSQL vía Prisma.

3. El Viaje de una Funcionalidad: Usuario a Socio

Analicemos el flujo de alta y promoción con código comentado paso a paso:

Paso 1.1: El DTO de Entrada (crear-usuario.dto.ts)

Explicación didáctica: Los decoradores de class-validator aseguran que los datos cumplan las reglas antes de entrar al sistema. El campo contrasenia se marca como write-only para proteger las credenciales.

```
export class CrearUsuarioDto {
  @ApiProperty({ enum: ROLES_ALTA }) @IsIn(ROLES_ALTA)
  rol!: RolAlta; // Rol inicial: solo EXTERNO, RECEPCION o GERENTE

  @ApiProperty({ example: '35123456' }) @Matches(/^([0-9])+$/)
  dni!: string; // Garantiza que el DNI contenga solo dígitos

  @ApiProperty() @MinLength(8)
  contrasenia!: string; // Longitud mínima de seguridad exigida
}
```

Paso 1.2: El Controlador (usuarios.controller.ts)

Explicación didáctica: Maneja el protocolo HTTP. Devuelve el recurso creado e inyecta la cabecera estándar REST Location indicando dónde consultar el nuevo usuario.

```
@Post()
async crear(@Body() dto: CrearUsuarioDto, @Res({ passthrough: true }) res: Response) {
  const usuario = await this.usuariosService.crear(dto);
  res.setHeader('Location', `/api/v1/usuarios/${usuario.id}`); // Estándar REST
  return usuario;
}
```

Paso 1.3: El Servicio y Seguridad (usuarios.service.ts)

Explicación didáctica: Valida la unicidad en la base de datos para evitar duplicados y aplica un algoritmo de hash con salting (bcrypt) para nunca almacenar contraseñas en texto plano.

```
async crear(dto: CrearUsuarioDto): Promise<UsuarioOutDto> {
  const duplicado = await this.usuarios.buscarPorDniOEmail(dto.dni, dto.email);
  if (duplicado) throw new ConflictException('DNI o email duplicado.');
```



```
  const hash = await bcrypt.hash(dto.contrasenia, await bcrypt.genSalt(10));
  const nuevo = await this.usuarios.crear({ ...dto, contrasenia: hash });
  return this.aOut(nuevo); // Mapea a DTO de salida ocultando el hash
}
```

Seguridad: UsuarioOutDto

El uso de @Exclude() y @Expose() en los DTOs de salida garantiza que la contraseña nunca sea devuelta en las respuestas JSON del servidor.

Paso 2: Promoción a Socio y Transacciones (prisma-socio.repository.ts)

Explicación didáctica: Un socio es una extensión 1:1 de un usuario. Para asegurar que la creación de la tabla Socio, la Membresía y la actualización del Rol ocurran de forma totalmente segura o se reviertan juntas, utilizamos una **transacción atómica** con Prisma.

```
async crear(socio: SocioNuevo): Promise<Socio> {
  return this.prisma.$transaction(async (tx) => {
    // 1. Crea el registro de socio vinculado al usuario
    const nuevoSocio = await tx.socio.create({ data: { usuario_id: socio.usuario_id, sede_id:
socio.sede_id } });
    // 2. Si especifica plan, crea su membresía vigente
    if (socio.plan) await tx.membresia.create({ data: { socio_id: nuevoSocio.id, plan:
socio.plan } });
    // 3. Actualiza el rol del usuario base a SOCIO
    await tx.usuario.update({ where: { id: socio.usuario_id }, data: { rol: 'SOCIO' } });
    return this.aDominio(nuevoSocio);
  });
}
```

Paso 3: Asignación de Membresía y Entidades de Dominio

Explicación didáctica: Las reglas de cálculo temporal residen en la entidad pura, mientras que el servicio valida las precondiciones antes de registrar la membresía.

```
// Lógica pura de dominio en entidades
export function calcularVigencia(plan: PlanMembresia, desde = new Date()) {
  const fecha_fin = new Date(desde);
  if (plan === 'MENSUAL') fecha_fin.setMonth(fecha_fin.getMonth() + 1);
  if (plan === 'ANUAL') fecha_fin.setFullYear(fecha_fin.getFullYear() + 1);
  return { fecha_inicio: desde, fecha_fin };
}
```

Paso 4: Baja de Socio y Preservación Histórica

Explicación didáctica: Al dar de baja a un socio, **no se borra el usuario** para no corromper reportes de pagos o reservas históricas. Solo se elimina la membresía y se retorna el rol a EXTERNO.

```
async eliminar(id: number): Promise<void> {
  await this.prisma.$transaction(async (tx) => {
    const socio = await tx.socio.findUnique({ where: { id } });
    if (!socio) return;
    await tx.membresia.deleteMany({ where: { socio_id: id } });
    await tx.socio.delete({ where: { id } });
    await tx.usuario.update({ where: { id: socio.usuario_id }, data: { rol: 'EXTERNO' } });
  });
}
```

4. Estandarización de Errores: Norma RFC 9457

Explicación didáctica: El filtro global intercepta excepciones de Prisma y genera respuestas estandarizadas application/problem+json para que los clientes interpreten los errores sin ambigüedades.

```
{
  "type": "about:blank", "title": "Conflicto en recurso",
  "status": 409, "detail": "El socio ya cuenta con una membresía activa."
}
```

5. Inyección de Dependencias (DI) en NestJS

Explicación didáctica: Vinculamos los tokens abstractos con las clases concretas de Prisma mediante el módulo, desacoplando la lógica de negocio de la infraestructura de persistencia.

```
@Module({
  controllers: [UsuariosController, SociosController, MembresiasController],
  providers: [
    { provide: USUARIO_REPOSITORY, useClass: PrismaUsuarioRepository }, // Inversión de DIP
    { provide: SOCIO_REPOSITORY, useClass: PrismaSocioRepository },
    UsuariosService, SociosService, MembresiasService,
  ],
})
export class UsuariosModule {}
```

6. Próximos Pasos y Preguntas Frecuentes

- **M2 a M4 (Gimnasio, Clases, Canchas):** Replicarán este mismo patrón en capas.
- **M5 Pagos:** Utilizará Arquitectura Hexagonal para aislar pasarelas de pago.

Q1: ¿Por qué Membresía tiene relación 1:1 con Socio y no 1:N?

Simplifica el estado vigente actual en una única fila. Las renovaciones extienden la fecha de fin y los pagos históricos se guardan en la tabla analítica `PagoMembresia`.

Q2: ¿Por qué se inyecta por token en vez de usar Prisma directamente?

Permite cumplir con el principio DIP, facilitando la creación de mocks para pruebas unitarias sin depender de una base de datos real.